

VIZINHANÇA E BUSCA LOCAL

DCE770 - Heurísticas e Metaheurísticas

Atualizado em: 26 de agosto de 2026

Iago Carvalho

Departamento de Ciência da Computação



DA CONSTRUÇÃO AO REFINAMENTO

Nas aulas anteriores, utilizamos heurísticas construtivas para gerar soluções rapidamente

- Em geral, decisões gulosas eram tomadas uma única vez
- Também estudamos mecanismos para tratar restrições e obter soluções viáveis

Entretanto, a visão míope de uma construção gulosa frequentemente produz soluções subótimas

Agora, partiremos de uma solução já construída e tentaremos **refiná-la**

Seja s a solução produzida por uma heurística construtiva

Uma busca local repete o seguinte ciclo:

1. Produzir soluções próximas de s , chamadas de *soluções vizinhas*
2. Verificar a viabilidade e avaliar a qualidade desses candidatos
3. Escolher um movimento aceitável de s para algum vizinho s'
4. Atualizar a solução corrente e repetir o processo

Assim, fazemos uma amostragem guiada do espaço de busca Ω

COMPONENTES DE UMA BUSCA LOCAL

Uma heurística de busca local combina:

- Uma solução inicial s
- Uma ou mais estruturas de vizinhança N
- Operadores que produzem movimentos em Ω
- Uma função objetivo f e, quando possível, avaliações incrementais
- Uma regra de aceitação dos movimentos
- Um critério de parada

O desempenho depende tanto da vizinhança quanto da maneira como ela é explorada

Vamos distinguir dois conjuntos:

Ω : espaço de busca e $\mathcal{F} \subseteq \Omega$: soluções viáveis

Um movimento aplicado a $s \in \mathcal{F}$ pode produzir $s' \in \mathcal{F}$ ou $s' \notin \mathcal{F}$

Quando um vizinho é inviável, podemos:

- Rejeitá-lo ou utilizar apenas movimentos que preservem a viabilidade
- Repará-lo antes da avaliação
- Admiti-lo temporariamente e penalizar as violações

Uma estrutura de vizinhança é uma função

$$N : \Omega \rightarrow 2^{\Omega}, \quad s \mapsto N(s).$$

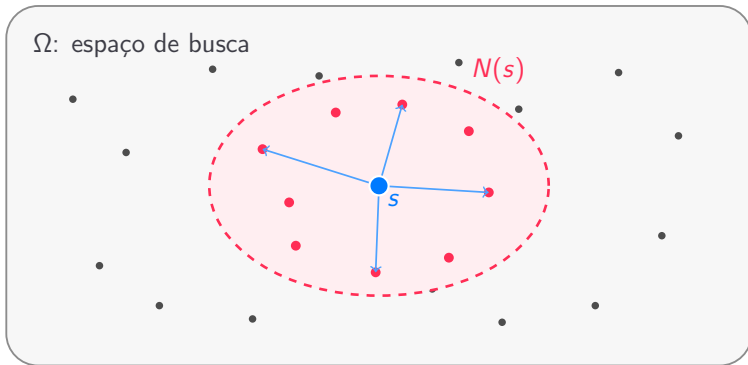
$N(s)$ é o conjunto de soluções que podem ser alcançadas a partir de s por um movimento

Uma solução s' é vizinha de s se, e somente se, $s' \in N(s)$

Se a busca considera apenas soluções viáveis, podemos utilizar

$$N_{\mathcal{F}}(s) = N(s) \cap \mathcal{F}.$$

VIZINHANÇA NO ESPAÇO DE BUSCA



A noção de “proximidade” não precisa ser geométrica: ela é determinada pelos movimentos permitidos.

Um *operador de vizinhança* implementa um **movimento** em Ω

Uma família de operadores $\mathcal{M}$ pode induzir a vizinhança

$$N(s) = \{m(s) : m \in \mathcal{M} \text{ e } m \text{ é aplicável a } s\}.$$

- O operador determina quais soluções podem ser alcançadas em um passo
- A regra de aceitação determina se o movimento será realizado
- Aplicar movimentos arbitrariamente não garante melhoria nem convergência

O QUE CARACTERIZA UMA BOA VIZINHANÇA?

Uma estrutura de vizinhança deve equilibrar:

- **Localidade:** pequenas alterações devem produzir soluções relacionadas
- **Alcance:** movimentos sucessivos devem explorar regiões relevantes de Ω
- **Viabilidade:** deve ser possível testar, preservar ou recuperar as restrições
- **Tamanho:** vizinhanças muito grandes podem ser caras de explorar
- **Avaliação:** o ganho de um movimento deve ser calculado eficientemente

Por isso, uma mesma heurística pode combinar várias vizinhanças

Considere um problema de minimização e uma solução $s^* \in \mathcal{F}$

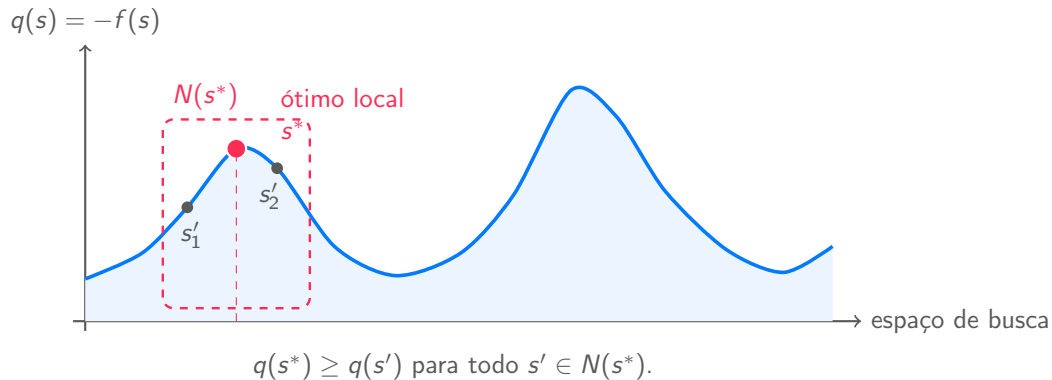
s^* é um **ótimo local em relação a N** quando

$$f(s^*) \leq f(s'), \quad \forall s' \in N(s^*) \cap \mathcal{F}.$$

- Não existe vizinho viável estritamente aprimorante
- Uma solução pode ser ótima para uma vizinhança e não ser para outra
- Um ótimo local não precisa ser globalmente ótimo

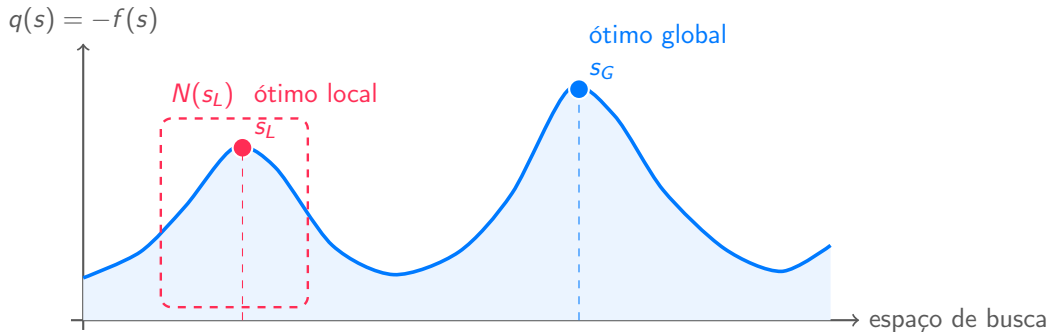
ÓTIMO LOCAL

Na figura, a altura representa a qualidade $q(s) = -f(s)$: quanto maior, melhor.



ÓTIMO LOCAL E GLOBAL

Na figura, a altura representa a qualidade $q(s) = -f(s)$: quanto maior, melhor.



Para atingir s_G , a busca precisaria aceitar uma redução temporária de $q(s)$.

A SOLUÇÃO INICIAL IMPORTA

Uma busca local é um método de **intensificação**: ela explora profundamente a região próxima da solução inicial

Com a mesma função objetivo e a mesma vizinhança, duas partidas podem levar a ótimos locais diferentes:

$$s_0^{(1)} \rightsquigarrow s_L^{(1)} \quad \text{e} \quad s_0^{(2)} \rightsquigarrow s_L^{(2)}.$$

- A qualidade da heurística construtiva influencia a região inicialmente explorada
- Uma boa solução inicial não garante o melhor ótimo local
- Reinícios a partir de soluções diferentes aumentam a diversidade da busca

Duas soluções s_1 e s_2 podem ou não ser vizinhas

- Essa relação depende da estrutura de vizinhança escolhida

Nesta aula, estudaremos métodos de **trajetória**, que mantêm uma solução corrente

- A solução inicial é frequentemente produzida por uma heurística construtiva
- Operadores de vizinhança geram candidatos ao redor da solução corrente
- Uma regra de aceitação decide para qual solução a busca se moverá

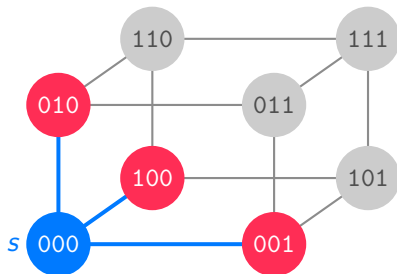
ESQUEMAS DE VIZINHANÇA CLÁSSICOS

VIZINHANÇA NO ESPAÇO $\{0, 1\}^3$

Considere soluções binárias e movimentos que alteram exatamente um bit

$$N_1(s) = \{s' \in \{0, 1\}^3 : d_H(s, s') = 1\},$$

onde d_H é a distância de Hamming



$N_1(s) = \{001, 010, 100\}$
um bit é alterado por movimento

Cada solução possui três vizinhos nessa estrutura.

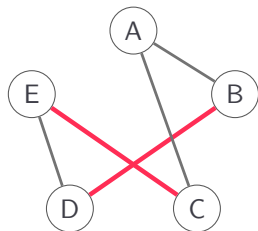
Uma das vizinhanças mais tradicionais para o problema do caixeiro viajante é a 2-opt

1. Selecione duas arestas não adjacentes na rota atual do caixeiro
2. Remova essas arestas, gerando dois caminhos disjuntos
3. Reconecte os caminhos invertendo um segmento da rota

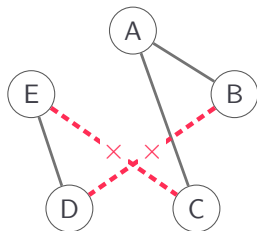
No PCV euclidiano, esse movimento pode *descruzar* arestas

Para uma rota simétrica com n vértices, existem $n(n-3)/2 = \Theta(n^2)$ movimentos 2-opt

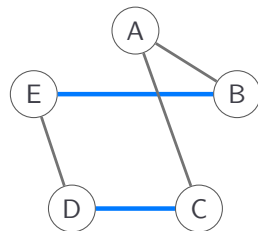
PCV: MOVIMENTO 2-OPT



Rota atual
arestas (B, D) e (E, C)



Remover duas arestas
dois caminhos são obtidos



Rota após 2-opt
novas arestas (B, E) e (D, C)

PCV: CÁLCULO INCREMENTAL NO 2-OPT

No PCV simétrico, se o movimento remove (a, b) e (c, d) e adiciona (a, c) e (b, d) , então

$$\Delta f = c_{a,c} + c_{b,d} - c_{a,b} - c_{c,d}.$$

Para minimização, o movimento é aprimorante quando $\Delta f < 0$

Exemplo: se os custos removidos são 13 e 12, e os adicionados são 7 e 8,

$$\Delta f = (7 + 8) - (13 + 12) = -10.$$

- Avaliar um movimento requer apenas quatro arestas: $\mathcal{O}(1)$
- Examinar toda a vizinhança requer $\mathcal{O}(n^2)$ avaliações

PROBLEMA DE ROTEAMENTO DE VEÍCULOS

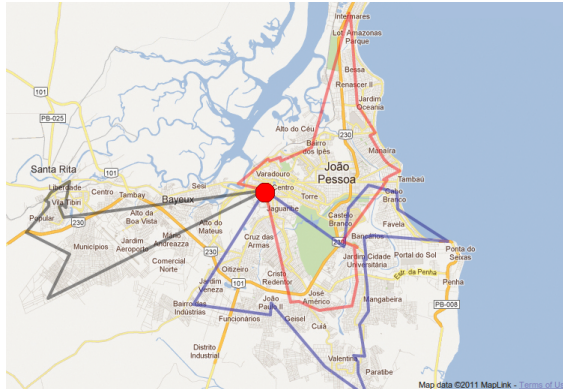
O problema de roteamento de veículos generaliza o PCV para múltiplas rotas

Na versão capacitada, cada cliente i possui demanda q_i e cada veículo possui capacidade Q

Devemos encontrar um conjunto de rotas de menor custo tal que:

- Cada rota começa e termina no depósito
- Cada cliente é atendido exatamente uma vez
- A carga de cada rota respeita $\sum_{i \in r} q_i \leq Q$
- O custo total das rotas é minimizado

PROBLEMA DE ROTEAMENTO DE VEÍCULOS



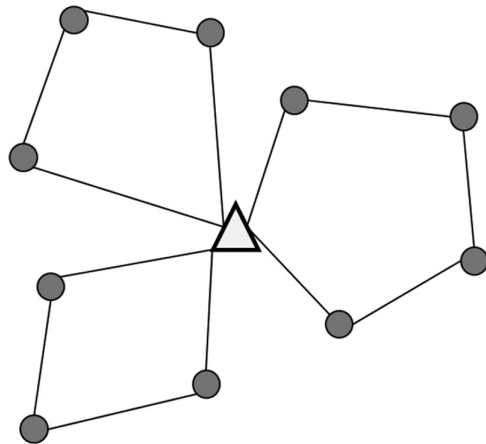
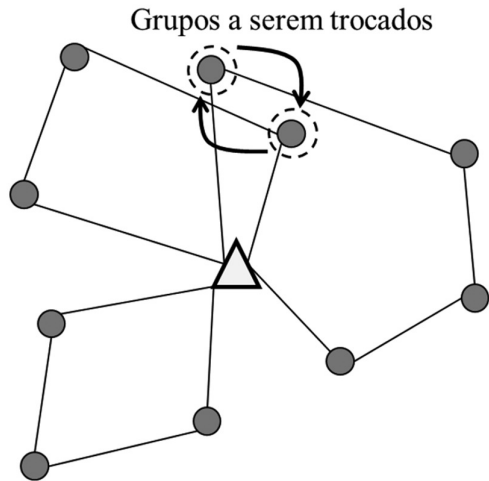
O depósito conecta várias rotas, e cada cliente pertence a exatamente uma delas.

Existem diversas estruturas de vizinhança para o problema de roteamento de veículos

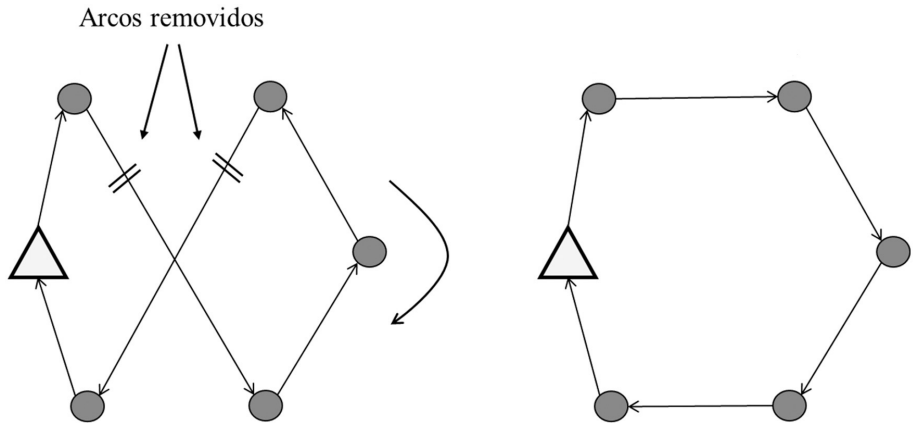
- **2-opt intra-rota**
 - Inverte um segmento dentro de uma única rota
- **2-opt* entre rotas**
 - Troca as extremidades de duas rotas
- **Swap(1,1)**: troca dois clientes de rotas distintas
- **Relocate/shift**: transfere um cliente para outra posição ou rota
- **Or-opt**: transfere uma sequência de clientes

Todo movimento inter-rotas deve verificar novamente as capacidades

PRV: MOVIMENTO SWAP(1,1)



PRV: MOVIMENTO 2-OPT INTRA-ROTA



ALGORITMOS DE BUSCA LOCAL

BUSCA LOCAL POR DESCIDA

Para um problema de minimização, uma busca por descida aceita apenas movimentos estritamente aprimorantes

A partir de uma solução inicial $s \in \mathcal{F}$:

1. Explore a vizinhança viável $N_{\mathcal{F}}(s)$
2. Escolha algum vizinho s' com $f(s') < f(s)$
3. Atualize $s \leftarrow s'$ e reinicie a exploração

A busca termina quando nenhum vizinho viável aprimorante é encontrado

- Nesse momento, s é um ótimo local em relação a N

POR QUE A BUSCA POR DESCIDA TERMINA?

Suponha que $\mathcal{F}$ seja finito e que somente movimentos estritamente aprimorantes sejam aceitos

$$f(s_0) > f(s_1) > f(s_2) > \dots$$

- A função objetivo diminui a cada movimento aceito
- A busca não pode visitar uma solução já encontrada
- Como existem finitas soluções viáveis, o processo termina após um número finito de movimentos
- Ao terminar, a solução corrente é um ótimo local para a vizinhança utilizada

BUSCA PELO MELHOR APRIMORANTE

Algoritmo 1: Busca local com melhor aprimorante

Entrada : $N_{\mathcal{F}}$ // Vizinhança restrita às soluções viáveis
Entrada : f // Função objetivo a ser minimizada
Entrada : $s \in \mathcal{F}$ // Solução inicial

```
1  $melhorou \leftarrow$  verdadeiro
2 enquanto  $melhorou$  faça
3    $melhorou \leftarrow$  falso
4    $s^* \leftarrow s$ 
5   para cada  $s' \in N_{\mathcal{F}}(s)$  faça
6     se  $f(s') < f(s^*)$  então
7        $s^* \leftarrow s'$ 
8   se  $f(s^*) < f(s)$  então
9      $s \leftarrow s^*$ 
10     $melhorou \leftarrow$  verdadeiro
11 retorne  $s$ 
```

MELHOR APRIMORANTE E PRIMEIRO APRIMORANTE

Na estratégia do **melhor aprimorante**:

- Todos os vizinhos são examinados a cada iteração
- É escolhido o vizinho com o menor valor da função objetivo

Na estratégia do **primeiro aprimorante**:

- A exploração é interrompida ao encontrar o primeiro vizinho melhor
- A solução é atualizada e a vizinhança é explorada novamente
- A ordem dos vizinhos pode alterar o tempo e o ótimo local encontrado

Escolher o melhor vizinho imediato não garante encontrar o melhor ótimo local ao final

BUSCA PELO PRIMEIRO APRIMORANTE

Algoritmo 2: Busca local com primeiro aprimorante

```
Entrada :  $N_{\mathcal{F}}$  // Vizinhaça restrita às soluções viáveis
Entrada :  $f$  // Função objetivo a ser minimizada
Entrada :  $s \in \mathcal{F}$  // Solução inicial

1  $melhorou \leftarrow$  verdadeiro
2 enquanto  $melhorou$  faça
3    $melhorou \leftarrow$  falso
4   para cada  $s' \in N_{\mathcal{F}}(s)$  faça
5     se  $f(s') < f(s)$  então
6        $s \leftarrow s'$ 
7        $melhorou \leftarrow$  verdadeiro
8     interrompa
9 retorne  $s$ 
```

Um **platô** é uma região em que vários vizinhos têm o mesmo valor de função objetivo

- Com melhoria estrita, a busca para ao chegar a um platô
- Aceitar movimentos neutros pode permitir alcançar uma região melhor
- Porém, movimentos neutros também podem fazer a busca circular entre soluções

Políticas usuais para lidar com esse caso:

- Rejeitar empates
- Limitar movimentos neutros e registrar soluções visitadas
- Aleatorizar a ordem dos movimentos, perturbar ou reiniciar a solução

ATIVIDADE: ESCOLHENDO UM MOVIMENTO

Considere uma solução $s \in \mathcal{F}$ e os movimentos abaixo:

Movimento	$\Delta f = f(s') - f(s)$	Viável?
m_1	-3	Sim
m_2	-7	Não
m_3	-5	Sim

1. Qual movimento é escolhido pelo primeiro aprimorante na ordem m_1, m_2, m_3 ?
2. Qual movimento é escolhido pelo melhor aprimorante viável?
3. Como m_2 poderia ser tratado sem simplesmente descartá-lo?

DISCUSSÃO DA ATIVIDADE

Para a ordem m_1, m_2, m_3 :

- O **primeiro aprimorante** escolhe m_1 , pois ele é o primeiro movimento viável com $\Delta f < 0$
- O **melhor aprimorante viável** escolhe m_3 , pois $\Delta f = -5$ é a maior redução de custo entre os movimentos viáveis
- O movimento m_2 pode ser rejeitado, reparado ou avaliado com uma penalização pela inviabilidade

A escolha depende da política de viabilidade e da regra de aceitação – não apenas do valor de Δf

Uma busca que aceita apenas melhorias:

- Depende da solução inicial e da vizinhança escolhida
- Pode terminar em ótimos locais de baixa qualidade
- Pode ter dificuldade em platôs do espaço de busca
- Pode gastar muito tempo examinando vizinhanças grandes

Metaheurísticas procuram superar essas limitações utilizando mecanismos como:

- Reinícios, perturbações, memória e aceitação controlada de pioras

PRÓXIMOS PASSOS: METAHEURÍSTICAS

Nas próximas aulas, veremos diferentes *frameworks* de busca local

Eles combinam os elementos estudados até aqui:

- Heurísticas construtivas para gerar soluções iniciais
- Tratamento de restrições para controlar a viabilidade
- Operadores de vizinhança para explorar Ω
- Estratégias para escapar de ótimos locais